EXTENDED ABSTRACT
ACTIVITY TRACKING WITH THE REACHY MINI ROBOT

Artificial Intelligence
Lectoraat Data Intelligence
Zuyd Hogeschool

Caio de Alcantara Santos (1234567890)
Cas Limpens (1234567890)
Donné Deguelle (1234567890)
Gustavo Gonçalves da Costa (1234567890)
Kanak Manjari (supervisor)
Maarteen Vaessen (supervisor)

2026/2027, Q1
01-01-2026

Abstract

Home-based exercise monitoring could reduce repeated hospital visits, but it requires reliable activity recognition in unconstrained environments. This project investigates whether the Reachy Mini robot can provide person detection and activity monitoring while giving feedback as an exercise buddy. A feasibility prototype was developed using a camera-to-pose-to-classifier pipeline. The system tracks a person, extracts COCO-17 keypoints, computes temporal features, and classifies activities with random forest inference. The shared pipeline can run both in simulation and on the physical robot. MoveNet int8 was selected for the robot because of its hardware constraints, and recognized actions trigger antenna responses. Evaluation used subject-separated public skeleton datasets, external datasets, recorded robot footage, and both full-body and waist-up framing. In real-time evaluation, wave, clapping, and jumping jacks were sufficiently reliable for demonstration. Overall, the results support the Reachy Mini as a feasible CPU-based platform for activity-monitoring research, while showing that this prototype is not a CMAS scorer or a clinically validated decision-support system.

1 Introduction

1.1 Background

The Research Centre for Data Intelligence conducts applied research into, among other themes, smart assistants and human-computer interaction. This project was carried out in the context of the centre's "Measuring at Home with AI" research direction. The long-term motivation for that research is to make exercise monitoring more accessible to children with juvenile dermatomyositis (JDM). These children currently perform exercises related to the Childhood Myositis Assessment Scale (CMAS) under the supervision of specialists at the Wilhelmina Children's Hospital (WKZ). Repeated travel to the hospital is burdensome for children and their families, which creates an opportunity for technologies that can support monitoring at home. Automated home monitoring must do more than record a video. It must detect a person, estimate body movement from visual input, distinguish between activities, and communicate its interpretation clearly to the user. It must also operate under practical conditions, such as changing distances, incomplete views of the body, and limited computing resources. The stakeholders for this feasibility study are the Research Centre for Data Intelligence, the project client and researcher, and the children and families who represent the future users of a home-monitoring system. The project requirements were consolidated in a software requirements specification covering perception, robot interaction, activity recognition, feedback, session management, privacy, accuracy, reliability, usability, and safety. The current project addresses the technical feasibility of such a system; it does not develop or validate a medical assessment instrument.

1.2 Goal

The goal of this project is to investigate whether the Reachy Mini can perform person detection and activity monitoring, including expressive feedback, under the computational constraints of the wireless robot. The resulting artefact is a working prototype that runs in simulation and on the physical robot. It contains a shared camera-to-pose-to-classifier pipeline, temporal feature extraction, activity-recognition models, and robot applications that translate recognized actions into antenna responses. Candidate vision models were benchmarked under the robot's CPU constraints, leading to the selection of MoveNet int8 for on-robot pose estimation. Model-training and export tools were also developed so that the robot can perform inference with NumPy-only model files rather than requiring the full training environment. Recognition models were evaluated on public skeleton datasets with subject or footage-group separation, as well as on robot recordings and an external dataset. Users can also record personalized movements and create their own datasets for future recognition models. The prototype recognizes six classes: none, wave, pushup, squat, clapping, and jumping jacks. Evaluation focuses on technical feasibility using simulation, public skeleton datasets, recorded robot footage, an external test dataset, and different camera framings. The robot's camera should ideally capture the whole person, because waist-up framing removes information that is important for some activities. The prototype is not a CMAS scorer, is not clinically validated, and should not be used as a clinical decision-support system. The central research question is:

> To what extent can the Reachy Mini detect a person, recognize physiotherapy-related exercises, and provide expressive feedback while operating within its computational constraints?

The remainder of this report first discusses the theoretical foundations and related solution space. It then describes the prototype and its evaluation method, presents the results, discusses their implications and limitations, and concludes with directions for future development.

2 Theoretical Framework

Skeleton-based human activity recognition (HAR) describes a person as a sequence of body keypoints rather than raw pixels. This reduces each frame to a few dozen values and makes the input less sensitive to clothing, background and lighting. The most accurate skeleton classifiers are deep networks. They lead benchmarks such as NTU RGB+D, but they generally assume GPU hardware. A lighter, well-established alternative computes hand-crafted statistics over a sliding temporal window and classifies them with a random forest [1]. This approach is fast, works with modest amounts of data, keeps its features interpretable and is small enough to run on the device itself.

Existing work highlights two uncertainties for this approach. First, windowed activity data contains near-duplicate samples from the same person, so record-wise train/test splits overestimate performance; subject-wise validation is needed to approximate the real use case [2]. Second, public datasets are recorded in specific settings, so a model that scores well on one corpus may not transfer to footage from another, such as a robot's camera in a living room. The innovative character of this part of the work is therefore not the classifier itself. It is the

combination of training on published skeleton datasets, evaluating with subject separation, exporting the forest to a NumPy-only format and connecting it through the robot's SDK. As a result, recognition runs on the Reachy Mini's CPU and directly drives expressive feedback.

3 Methods

3.2 Requirements and software requirements specification

The functional and non-functional requirements were derived from the project description and meetings with the client. They were consolidated in a software requirements specification (SRS), which served as the reference for design and evaluation. The functional requirements cover perception and local processing, robot behaviour and interaction, activity and exercise recognition, assessment and feedback, and session management. The non-functional requirements cover privacy, recognition accuracy, reliability, usability and safety. The complete SRS document contains the detailed requirement statements and is included as an attachment to the article.

3.3 Pose-model research and selection

We reviewed candidate models for human pose detection and temporal activity analysis under the Reachy Mini's CPU and camera constraints. The candidates included BlazePose for fast 2D pose detection, ViTPose as a transformer-based 2D estimator, WHAM and CameraHMR for 3D motion or pose-and-shape estimation, MoveNet for lightweight 2D pose estimation, SAM2 for segmentation, PySlowFast as a video-classification framework, FSPose for fused-heatmap pose estimation, GraphEnet for graph-based pose estimation, YOLOPose for pose estimation using the YOLO object-detection framework, MoViNets for efficient video recognition, and TSM-MobileNetV2 for lightweight temporal modelling.

BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose, YOLOPose and MoveNet were benchmarked under comparable conditions. We recorded throughput and mean pose-inference time, while considering whether the model could run on the robot and produce the COCO-17 representation required by the downstream feature and recognition pipeline. The selected model was MoveNet Lightning int8 through LiteRT, because it provided the most suitable trade-off between speed, keypoint compatibility and deployment feasibility.

3.6 Training, export and inference

The recognition models were trained without recording or processing any video. We used COCO-17 skeletons published by OpenMMLab for NTU RGB+D 60 [3] (wave, clapping) and UCF101 [4] (pushup, squat, jumping_jacks). HMDB51 was used only as an external test set. All other dataset classes were labelled none. Each clip was resampled from 30 to approximately 10 FPS to match the robot camera. It was then windowed by the same feature code that runs live: 1.5 s windows with 9 features for the binary wave model, and 3 s windows with 34 features for the six-class action model. To reflect the table-mounted camera, every clip was also windowed with leg and hip keypoints removed.

Random forests were trained with scikit-learn [5]: 100 trees of depth 6 for the wave model and 300 class-balanced trees for the action model. They were evaluated with five-fold GroupKFold by subject or footage group, following [2], and all precision, recall and F1 scores come from out-of-fold predictions. Instead of taking the most probable class, we tuned a confidence floor for each class on these predictions; windows below the floor are labelled none. The final forests were fitted on all windows and exported to a compressed .npz file containing each tree's node arrays. On the robot, a NumPy-only implementation walks these trees and reproduces scikit-learn's predict_proba, so scikit-learn is not needed at runtime.

3.7 Robot integration (SDK and REST API)

The applications use the Reachy Mini Python SDK (reachy_mini 1.10.0, matched to the daemon version) for motion, and the daemon REST API (port 8000) for state management. Before any movement, camera or tracking use, a standard-library preflight checks, in order, network reachability, daemon and backend state, running apps and moves, motor mode, sleep pose, camera ownership and tracker frames. With auto-fix enabled, it enables the motors, plays the wake-up move and re-acquires the camera. It returns an explicit ready, not-ready or unreachable status. A context manager wraps preflight, SDK connection and clean-up (stopping head tracking and, optionally, returning to sleep), so every application starts and ends in a known state.

Because the vision loop runs at approximately 10 FPS while the daemon controls at 50 Hz, a dedicated thread streams set_target commands at 50 Hz. Each recognised action is

answered with a distinct antenna signature (amplitude, frequency, whole number of cycles ending at the neutral pose), so observers can tell which action was recognised. Head and body following use an integrating controller on the image error of the torso centre. Body yaw extends the reach of the head, which is limited to 60 degrees relative to the body.

Further integrations are: (i) the annotated camera view is served as an MJPEG stream, so the robot runs headless while being observed from a browser; (ii) a remote mode streams low-resolution frames (192×108) to a laptop GPU for pose inference and receives keypoints over HTTP, while session logic, speech and motion stay on the robot; and (iii) in the coach workflow the robot trains a personal model on its own CPU, running scikit-learn in a low-priority subprocess so camera and head-following loops stay responsive and a failed fit never overwrites the previous model. Deployment uses an isolated virtual environment that leaves the robot's system environments untouched.

4 Results

4.2 Pose-model performance

The benchmark compared BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose and MoveNet as candidate components for the perception pipeline. MoveNet Lightning int8 achieved an average of 23.80 FPS, with a mean pose-inference time of approximately 34 ms on the Reachy Mini. This was the most suitable result for the project's CPU-only deployment target. The model also produces the COCO-17 keypoints expected by the temporal feature and recognition code.

The comparison also exposed deployment constraints. BlazePose and ViTPose are useful comparison models, but the robot cannot use the available BlazePose MediaPipe binary because it requires AES instructions unavailable on the Reachy's Cortex-A72; ViTPose's detector-and-pose pipeline is intended for the laptop or simulation environment. MoViNet-A0 and TSM-MobileNetV2 were considered for temporal classification rather than selected as replacements for the keypoint-based pipeline. Consequently, MoveNet int8 was used for robot inference, while the shared downstream representation remained model-independent.

4.3 Wave recognition

On two recorded sessions with our own robot (416 windows), the rule-based detector reached F1 0.811 (precision 1.000, recall 0.682). The NTU-trained forest, which never saw these recordings, reached F1 0.951 and outperformed a forest trained on the sessions themselves (F1 0.884, historical only). On unseen NTU subjects, where the negatives include deliberately confusable gestures (salute, selfie and brushing hair), F1 was 0.556. On HMDB51 movie clips it was 0.065, a deliberately pessimistic external check.

4.4 Six-class action recognition

Table 1: Out-of-fold F1 on held-out subjects or footage groups. Waist-up framing is simulated by removing the hip, knee and ankle keypoints.

Class	Full body	Waist-up
none	0.902	0.855
wave	0.918	0.873
squat	0.922	0.803
clapping	0.898	0.899
jumping_jacks	0.887	0.861
pushup	0.381	0.137

These are in-domain scores. A source confound inflates wave and clapping, both taken from the studio-recorded NTU corpus. On our robot recordings wave drops to 0.581, the figure to quote for a robot in a room, and clapping has not yet been measured on robot footage. Squats are almost never false positives (precision 0.998) but miss about one repetition in seven (one in three waist-up). Push-ups (19 training groups, per-fold F1 0.06–0.71) are insufficiently measured and are not demonstrated. Without crop augmentation, waist-up F1 for wave, squat and clapping fell to 0.000.

4.5 Engineering results

- One model file serves both simulation and robot. The six-class model is 5.3 MB and the wave model 0.25 MB, with no scikit-learn dependency at runtime.
- On the robot, the full wave pipeline runs at 8.7 FPS: 34 ms pose estimation and 3 ms motion processing (features and forest). The camera (approximately 10 FPS) is the remaining bottleneck.
- Antenna gestures are rendered smoothly at 50 Hz independently of the vision rate. Head and body following reached 75 degrees of gaze, compared with 35 degrees for the head alone.
- Preflight turned the robot's frequent post-boot state (asleep, motors off and camera released) into an automatic fix rather than a runtime failure.

5 Discussion

Maximum of 400 words.

The discussion describes a critical view (reflection) of the author(s) on the work delivered as a whole:

- Which aspects have contributed positively to the results achieved and why?
- Which aspects have been limiting for the results achieved and why?

Aspects can be considered broadly here. For example, process choices (method), design decisions (results) or other (whether or not externally determined) aspects that have influenced the results

6 Conclusions

Maximum of 400 words. The conclusion describes the extent to which the original goal of the assignment has been met with the help of the achieved (factual, substantive) results. In addition, any recommendations for follow-up work are described:

- Which artefacts have/have not been realised?
- To what extent have any boundary conditions/requirements been met?
- To what extent can the results be used/used for the original purpose?
- What recommendations are made on the basis of this, both towards practice and any necessary follow-up research into previously carried out work?

References

[1] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324

[2] S. Saeb, L. Lonini, A. Jayaraman, D. C. Mohr, and K. P. Kording, "The need to approximate the use-case in clinical machine learning," *GigaScience*, vol. 6, no. 5, pp. 1–9, 2017. DOI: 10.1093/gigascience/gix019

[3] A. Shahroudy, J. Liu, T.-T. Ng, and G. Wang, "Ntu rgb+d: A large scale dataset for 3d human activity analysis," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 1010–1019. DOI: 10.1109/CVPR.2016.115

[4] K. Soomro, A. R. Zamir, and M. Shah, "Ucf101: A dataset of 101 human actions classes from videos in the wild," Center for Research in Computer Vision, University of Central Florida, Tech. Rep. CRCV-TR-12-01, 2012. arXiv: 1212.0402.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.

Appendices

A Additional material